

Test Assignment

GenAI Software Solutions Engineer

Objective

Design and implement a fully local AI desktop application capable of analyzing and querying short video files (approximately 1 minute each).

The application should extract and summarize content, generate reports (PDF/PPT), and operate entirely offline using local AI models and self-developed MCP servers.

Key Skills

Agentic AI · MCP · Python · JavaScript/React · Tauri · C# · OpenVINO · Local Model Runtimes · gRPC API

Functional Requirements

- Allow users to **select and upload local .mp4 files**.
- Support **natural language interaction to process and query video content**.
- Example user queries:
 - “Transcribe the video.”
 - “Create a PowerPoint with the key points discussed in the video.”
 - “What objects are shown in the video?”
 - “Are there any graphs in the video? If yes, describe them.”
 - “Summarize our discussion so far and generate a PDF.”
- **Human-in-the-loop clarification** for ambiguous queries or low-confidence multi-agent routing. Example:
 - “Did you mean <this> or <that>?”
 - “Can you provide more details?”
- Maintain **persistent chat history**, accessible for reading even after app restart.

Architecture Requirements

Frontend

- Built with **React + Tauri** for a lightweight desktop interface.
- Chat-style conversational UI.
- Local persistent storage for chat history.
- Communication with the backend via **gRPC**.

Backend

- **Python-based**.
- Should include **multiple Agents** communicating with **MCP servers**.

- Example Agents / MCP Servers (not required to be exact):
 - Transcription Agent: Performs speech-to-text extraction.
 - Vision Agent: Handles object recognition, captioning, and text/graph extraction.
 - Generation Agent: Creates PDF or PowerPoint summaries.
- All AI inference must run **locally**, using **OpenVINO-optimized** or **Hugging Face** models.
- No cloud connectivity or reliance on public MCP servers.

Bonus Points

- Well-structured, modular code with architecture diagrams and a detailed setup guide.
- A **C# launcher** for running the packaged backend executable (e.g., via PyInstaller).

Submission Guidelines

Submit a **GitHub repository** within **7 days** that includes:

- Complete source code (frontend and backend).
- Clear setup and execution instructions.
- Example input files, sample queries, and generated outputs (PDF/PPTX).
- A summary describing what works, what doesn't, potential improvements, encountered challenges, and what could be achieved with more time.